Concurrency

Event loop, isolates, and goroutines

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Spring 2027

TODAY

What you should leave with

Respect the frame budget

One frame is 1 ÷ refresh rate. Blocking the UI thread past that costs a dropped frame, not a crash.

Move CPU work off the UI thread

compute() for a one-shot call, Isolate.spawn for a worker that lives longer or streams results back.

Write goroutines and channels

Go's concurrency primitives: go func(), typed channels, and select, on the server side of the same stack.

Compare the two models

Dart isolates copy messages between heaps. Goroutines share one heap and rely on channels for discipline.

PART 1 · FOUNDATIONS

The frame budget and the event loop

One thread, one queue, and what running on it costs

FOUNDATIONS

One frame budget = 1 ÷ refresh rate

16.7 ms

60 Hz: older devices, power-saving mode

11.1 ms

90 Hz: common mid-range in 2026

8.3 ms

120 Hz: most 2026 flagships

Inside that budget the UI thread has to run build, layout and paint, then hand the frame to the GPU.

Miss the budget and the frame is dropped. Users see that as a single stutter; repeated overruns look like continuous jank.

Do not hard-code 16 ms. Phones change refresh rate at runtime to save power, so the budget itself moves.

The event loop and the microtask queue

Your app runs on a single thread inside the main isolate, and that isolate owns the UI.

That thread runs an event loop: run one event, drain the entire microtask queue, then take the next event.

`await` starts nothing itself. It registers a continuation and hands control back to the loop.

Nothing else runs meanwhile: a 400 ms function costs 400 ms of dropped frames.

1. Microtask queue: Future callbacks, drained completely first.
2. Event queue: I/O, timers, gestures, platform messages.
3. Frame: build, then layout, then paint.

Concurrent, not parallel. One thread, interleaving work that is mostly waiting.

Futures and async/await: a refresher

```dart
Future<User> loadUser(String id) async {
  final res = await dio.get('/users/$id');
  return User.fromJson(res.data);
}

Future<void> refresh() async {
  try {
    user = await loadUser(id);
  } finally {
    if (mounted) setState(() {});
  }
}
```

ADMD Lecture 4 owns this topic.
This slide is a reminder, not new material.
A failure inside async code completes the Future with an error instead of throwing at the call site. `await` rethrows it, so try/catch behaves exactly as it does in synchronous code.

Streams: many values, over time

```dart
// A Stream is many values, over time.
Stream<int> ticker() async* {
  for (var i = 0; ; i++) {
    await Future.delayed(const
    Duration(seconds: 1));
    yield i;
  }
}
```

A Future is one value, later. A Stream is many values, over time.

StreamBuilder subscribes, rebuilds on each event and cancels for you.

Cancel every subscription you create by hand, in `dispose()`.

One value gives a Future. Many values give a Stream. Both ride the same event loop.
→ WebSockets and sensor feeds in later lectures.

Async is not parallel

```
// This function is async. It still
// freezes the UI.
Future<int> sumTo(int n) async {
  var total = 0;
  for (var i = 0; i < n; i++) {
    total += i;   // pure CPU, no yield
  }
  return total;   // the loop never turned
}
```

`async` only helps when something else does the waiting: a socket, a disk, a timer.

Pure CPU work never returns to the event loop. Cooperative multitasking only works if every task yields.

Typical offenders: a large JSON decode, image resize, encryption, sorting a big list.

Rule of thumb. Waiting on I/O: `await`. Burning CPU for longer than a frame: an isolate.

PART 2 · ISOLATES

Moving CPU work off the UI thread

A second heap, not a second thread inside yours

compute(): the one-call isolate

```dart
import 'package:flutter/foundation.dart';

// Must be a top-level or static function.
List<Photo> parsePhotos(String body) {
  final list = jsonDecode(body) as List;
  return list.map(Photo.fromJson).toList();
}

Future<List<Photo>> fetchPhotos() async {
  final res = await http.get(url);   // I/O
  return compute(parsePhotos, res.body);
}
```

`compute()` spawns a short-lived isolate, runs one function, sends the result back and shuts it down.

Arguments and results must be sendable: primitives, Strings, Lists and Maps of them.

It costs a few milliseconds to spawn and copies the payload both ways. Do not reach for it to save 2 ms.

Isolate.spawn, SendPort, ReceivePort

```dart
Future<int> sumInIsolate(int n) async {
  final inbox = ReceivePort();
  await Isolate.spawn(_worker,
    [inbox.sendPort, n]);
  return await inbox.first as int;
}

void _worker(List msg) {
  Isolate.exit(msg[0], sumTo(msg[1] as int));
}
```

ReceivePort is your inbox. SendPort is a handle other isolates can post to.

Messages are deep-copied between heaps: nothing is shared, so nothing needs a lock.

`Isolate.exit` hands the result over without copying it, then ends the isolate. `sumTo` is the loop from the previous slide.

Async vs isolates, precisely

	ASYNC / AWAIT	ISOLATE
Threads	one, on the UI thread	one per isolate, scheduled by the OS
Memory	one heap, shared by everything	a separate heap per isolate
Talking	just call the function	messages over ports, copied
Good for	I/O: network, disk, database, timers	CPU: parse, decode, encrypt, sort
Data races	impossible: one thing runs at a time	impossible: no shared memory
Ordering races	possible	possible: replies can reorder

No shared memory means no data races. It does not mean no races. A reply can still arrive out of order, and isolate count is not capped by cores.

PART 3 · GOROUTINES

Go’s concurrency model

Shared memory, channels, and one goroutine per request

Why Go for the server side of this stack

One static binary. `go build` produces one executable, no runtime and no dependency folder to ship alongside it.

It starts in milliseconds. Nothing to boot means short cold starts on a scale-to-zero platform.

The standard library is already the server. `net/http` and `encoding/json` need no framework.

The goroutines this section covers are one per request, by default, in that same standard library.

```
CGO_ENABLED=0 GOOS=linux \
  GOARCH=arm64 \
  go build -o server
    ./cmd/server

scp server
    root@vps:/usr/local/bin/
ssh root@vps 'systemctl
    restart api'
```

Same stack, a different runtime

On the phone

One thread owns the UI, and it has a frame budget to protect. Concurrency exists to keep that one thread free.

On the server

No frame to protect, only requests to serve. Concurrency exists to handle many of them at once, on every core you have.

The course stack is Flutter and Dart on the phone, Go on the server. Same problem, concurrency, two different reasons to reach for it.

Go's answer is the goroutine: a function that runs concurrently with the rest of the program, cheap enough to start one per incoming request.

A Go handler: one goroutine per request

```
type Reading struct {
    DeviceID string  `json:"device_id"`
    TempC    float64 `json:"temp_c"`
}
func handleReading(w http.ResponseWriter,
    r *http.Request) {
    var in Reading
    json.NewDecoder(r.Body).Decode(&in)
    w.WriteHeader(http.StatusCreated)
}
```

`net/http` hands every connection to its own goroutine automatically.

`handleReading` already runs concurrently for every caller. Nothing here asked for that.

`"POST /readings"` routing is built into the standard library, no framework needed.

The goroutine is what this section explains next.

Goroutines: cheap, runtime-scheduled

```
func main() {
    for i := 0; i < 5; i++ {
        go worker(i)
    }
    time.Sleep(time.Second)
}

func worker(id int) {
    fmt.Println("worker", id)
}
```

`go` returns at once; it never waits for the call.

A goroutine starts at 2 KB of stack. An OS thread starts at megabytes.

The runtime spreads goroutines over a few OS threads, one per core.

Cheap enough to have thousands. An idle goroutine costs almost nothing.

Waiting for many goroutines: sync.WaitGroup

```
var wg sync.WaitGroup
for i := 0; i < 5; i++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()
        process(id)
    }(i)
}
wg.Wait()
```

`Add` before the goroutine starts, `Done` when it finishes, usually deferred.

`Wait` blocks the caller until the count reaches zero.

Passing `id` as an argument avoids every goroutine capturing the same loop variable.

This replaces the Sleep from the previous slide. Wait for the actual work to finish, not for a guessed amount of time.

Channels: typed, synchronized queues

```
ch := make(chan int) // unbuffered
go func() {
    ch <- 42 // blocks until read
}()
value := <-ch // blocks until sent
close(ch) // no more sends
```

A channel is a typed pipe between goroutines. Send and receive block by default, and that is what synchronizes them.

Closing a channel signals “no more values”. A receive from a closed channel returns immediately.

Only the sender should close a channel.

Rob Pike, Go proverb. “Don’t communicate by sharing memory; share memory by communicating.”

Buffered channels and backpressure

```
ch := make(chan int, 3) // cap 3
ch <- 1
ch <- 2
ch <- 3 // buffer not full yet
ch <- 4 // buffer full: blocks
```

A buffered channel decouples sender and receiver up to *N* values.

Send blocks only once the buffer is full. Receive blocks only once it is empty.

Picking *N* is a capacity decision, the same trade-off as sizing any queue.

A bounded queue, for free. No separate library: the channel itself gives you backpressure.

Goroutine and channel pitfalls

PITFALL	WHAT HAPPENS	FIX
Deadlock	Every goroutine blocks on a channel with no partner to unblock it	Add a receiver, a buffer, or a `select` with a default
Goroutine leak	A goroutine blocks forever on a channel nobody sends on or closes	Pass a context and select on its `Done()` channel
Data race	Two goroutines read and write one variable with no channel or mutex	Run `go run -race`, or guard the variable with `sync.Mutex`

None of this is checked for you. The Dart isolate table two sections back had two rows of “impossible”. Go has none: shared memory buys speed, and hands the safety work back to you.

select: waiting on multiple channels

```
select {
case v := <-ch1:
    fmt.Println("from ch1:", v)
case v := <-ch2:
    fmt.Println("from ch2:", v)
}
```

`select` blocks until one of its cases can proceed.

If several are ready at once, Go picks one at random, so no single channel is starved.

Dart has no direct equivalent: the closest is racing two Futures with a library helper.

select with a timeout and a default case

```
select {
case v := <-ch:
    fmt.Println(v)
case <-time.After(2 * time.Second):
    fmt.Println("timed out")
default:
    fmt.Println("nothing ready yet")
}
```

`time.After` returns a channel that fires once, after the given duration: racing it against `ch` is a timeout.

`default` makes the whole `select` non-blocking: nothing ready, run this branch and move on.

Never combine a real case with `default` unless you mean “check once and give up”.

select turns waiting into a choice. Exactly one branch runs; nothing else blocks the goroutine.

PART 4 · COMPARED

Producer and consumer, Dart and Go

Same shape, opposite guarantee

COMPARED

Producer/consumer in Dart

```
final inbox = ReceivePort();
await Isolate.spawn(producer, inbox.sendPort);
await for (final item in inbox) {   // consumer
  if (item == null) break;
  print(item);
}
void producer(SendPort out) {
  for (var i = 0; i < 5; i++) out.send(i);
  out.send(null);
}
```

The two heaps never touch. Every value crossing `inbox` is copied. `null` is this example's own end-of-stream marker; the loop reads it, the language does not.

COMPARED

Producer/consumer in Go

```
ch := make(chan int)
go func() {            // producer
    for i := 0; i < 5; i++ {
        ch <- i
    }
    close(ch)
}()
for item := range ch { // consumer
    fmt.Println(item)
}
```

The heap is shared. Nothing stops another goroutine from reading `ch` too: a convention, not a guarantee.

COMPARED

Dart isolates vs Go goroutines

	DART ISOLATE	GO GOROUTINE
Memory	a separate heap per isolate	one heap, shared by every goroutine
Talking	ports; messages are copied	channels; you choose what crosses
Start-up cost	a few ms, a fresh heap	about 2 KB of stack, growing
Typical count	dozens, spawned deliberately	thousands, one per connection
Data races	impossible: no shared memory	possible: guard it yourself
On failure	ends that isolate only	an unrecovered panic ends the process

Same task, opposite guarantee. Isolates trade a copy per message for safety. Goroutines trade the copy for speed, and hand the discipline back to you.

COMPARED

What each model buys you: phone vs server

On the phone

Protect one frame budget. Anything over a few milliseconds of CPU work moves to an isolate; you spawn only as many as the work actually needs.

On the server

Maximize throughput, with no frame to protect. One goroutine per request is the default because it is cheap, spread across every core with no extra code from you.

Concurrency solves a different problem on each side. The phone spends it protecting a deadline. The server spends it on throughput. → Lecture 10 puts a whole Go backend on top of this model.

RECAP

Three things to remember

1. One frame budget is 1 divided by refresh rate. The event loop runs one thing at a time and drains every microtask before the next event.
2. compute() and Isolate.spawn move CPU work to a separate heap. Messages are copied, so data races are impossible, but ordering races are not.
3. Goroutines share one heap and are synchronized with channels and select. Cheap enough to use one per request, but a race or a panic is now your job to prevent.

FURTHER READING

dart.dev/language/concurrency · go.dev/tour/concurrency · go.dev/doc/effective_go · api.flutter.dev: Isolate

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel